

실행 · 운영 관점 비교

vLLM vs furiosa-llm

같은 OpenAI 호환 API, 정반대 실행 모델 — 설치 · 빌드 · serve · 호출 · 튜닝을 실제 명령어로 비교

furiosa-llm 2026.2.0 · vLLM 0.10.0 (V1) · 설치본 소스 + repo 실제 스크립트 기반

한 줄 차이 — 2 단계 AOT vs 1 단계 JIT

furiosa 는 먼저 컴파일해 “아티팩트”를 만든 뒤 그걸 띄웁니다. vLLM 은 띄우는 순간 로드 · 컴파일까지 한 번에.

furiosa-llm



느린 오프라인 빌드 1 회 → 저장된 바이너리 로드 (빠름) · shape·tp· 양자화가 빌드에 “박힘”

vLLM



별도 빌드 단계 없음 → 매 기동마다 가중치 로드 · JIT 컴파일 (시작 느림) · tp· 양자화 · shape 를 기동마다 자유 선택

설치와 전제 — 전용 SDK vs pip 한 줄

furiosa-llm

RNGD NPU 전용

설치 (Furiosa SDK 저장소)

```
# NPU 드라이버 · 펌웨어 · 컴파일러 선설치 필요
pip install furiosa-llm \
  --extra-index-url <furiosa-pypi>
furiosa-smi info # NPU 인식 확인
```

- torch 2.10 · transformers 5.1 등 버전 고정 (pin)
- furiosa-native-runtime / -compiler 동반 설치
- NPU 카드 + 드라이버 없으면 serve 불가

vLLM

NVIDIA GPU (CUDA)

설치 (PyPI)

```
# CUDA 런타임만 있으면 됨
pip install vllm

nvidia-smi # GPU 인식 확인
```

- PyPI 한 줄 (+ROCm/TPU/CPU 등 다중 백엔드 휠)
- 임의 HF 체크포인트 즉시 사용 가능
- GPU 만 있으면 어디서나 동일하게 동작

모델 준비 — furiosa 만 “빌드”가 필요

vLLM 에는 이 단계 자체가 없습니다. 그래서 furiosa 는 첫 준비가 무겁고, 대신 serve 가 가볍습니다.

furiosa-llm

AOT 컴파일 → 아티팩트

furiosa-llm build

```
furiosa-llm build Qwen/Qwen3-32B-FP8 \
./artifacts/qwen3-32b-fp8-tp8 \
-tp 8 \      # PE 8 개 = 카드 1 장
--prefill-buckets ... --decode-buckets ...
# 수 분 ~ 수십 분 ( 그래프 분할 + 컴파일 )
```

산출물 : artifact.json + EDF 바이너리 + param.safetensors + tokenizer → 디스크에 저장,
재사용

tp · 양자화 · shape (버킷) 가 여기서 “박힘”

→ 나중에 바꾸려면 이 build 를 다시 돌려야 함

vLLM

준비 단계 없음

(해당 없음)

모델 경로 (HF id) 를 serve 에 바로 넘기면 됩니다.
가중치 로드 · 컴파일은 모두 기동 시점에 일어납니다.

장점 vs 비용

+ 사전 빌드 필요 없음 — 어떤 모델이든 즉시 시도

+ tp · 양자화 · shape 를 기동마다 자유 변경

- 대신 매 기동마다 컴파일 / 그래프 캡처 비용 발생

3. 서버 실행

서버 실행 — 양쪽 다 OpenAI 호환 HTTP 서버

명령 모양은 닮았지만, furiosa 는 “아티팩트 경로 + --devices”, vLLM 은 “HF id + GPU” 입니다.

furiosa-llm serve

입력 = 빌드된 아티팩트

실제 사용 예 (run_rngd.sh)

```
furiosa-llm serve ./artifacts/qwen3-32b-fp8-tp8\  
--devices npu:0 \ # 어느 NPU 카드  
--host 0.0.0.0 --port 8000\  
--max-concurrency 32\  
--reasoning-parser qwen3\  
--enable-prefix-caching
```

vllm serve

입력 = HF 모델 id

실제 사용 예 (runners/server.py + models.yaml)

```
CUDA_VISIBLE_DEVICES=2 \ # 어느 GPU  
vllm serve Qwen/Qwen3-32B\  
--host 0.0.0.0 --port 8000\  
--max-num-seqs 32\  
--tensor-parallel-size 2 \ # 기동 시 결정  
--enable-prefix-caching
```

입력이 “모델 이름”이 아니라 “빌드 산출물 폴더” · NPU 카드는 --devices npu:N 으로 지정

입력이 “HF 모델 id”(없으면 자동 다운로드) · GPU 는 CUDA_VISIBLE_DEVICES 로 지정

공통: 둘 다 기동 후 [GET /v1/models](#) 로 헬스 체크 → 같은 OpenAI HTTP 엔드포인트 (/v1/chat/completions 등) 제공

코드로 직접 호출 — from ... import LLM, SamplingParams

클래스 이름 · 메서드까지 닮았습니다. 차이는 LLM() 에 “무엇을 넘기느냐”입니다.

furiosa-llm

LLM(아티팩트 경로)

```
from furiosa_llm import LLM, SamplingParams

# 인자 = 빌드된 아티팩트 경로
llm = LLM("./artifacts/qwen3-32b-fp8-tp8",
          devices="npu:0")
sp = SamplingParams(temperature=0.7,
                    max_tokens=256)
out = llm.generate(" 프롬프트 ", sp)
# (HF id 로 쓰려면 fxb= 로 v3 엔진 경로 )
```

vLLM

LLM(model=HF id)

```
from vllm import LLM, SamplingParams

# 인자 = HF 모델 id ( 런타임 로드 )
llm = LLM(model="Qwen/Qwen3-32B",
          tensor_parallel_size=2)
sp = SamplingParams(temperature=0.7,
                    max_tokens=256)
out = llm.generate([" 프롬프트 "], sp)
# tp·양자화 등을 생성자 인자로 바로 지정
```

클라이언트는 동일 — 단, furiosa 는 일부 필드를 조용히 버림

furiosa 서버 코드 상단에 “Adapted from vLLM” 이 명시돼 있을 만큼 API 표면이 같습니다.

동일한 OpenAI 클라이언트 — 서버만 바꾸면 됨 (base_url)

```
from openai import OpenAI
client = OpenAI(base_url="http://HOST:8000/v1", api_key="-") # furiosa·vLLM 동일
client.chat.completions.create(model=MODEL, messages=[...], temperature=0.7, stream=True)
```

양쪽 모두 그대로 동작

model · messages/prompt · temperature · top_p · top_k ·
max_tokens · stop_token_ids · stream · logprobs · 구조화 출력
(JSON/grammar)

furiosa 가 받지만 “무시”하는 필드

presence_penalty · frequency_penalty · seed · logit_bias · stop(
문자열)

→ 요청은 200 OK 지만 효과 없음 (to_sampling_params 에서 드롭). n>1 ·
투기적 디코딩은 아예 미지원.

튜닝 노브 대응표 — 이름까지 일부러 맞춰 놓음

벤치 비교용으로 정렬해 둔 매핑 (models.yaml) 을 그대로 옮긴 표입니다 .

항목	furiosa-llm	vLLM	비고
동시 처리 시퀀스 수	--max-concurrency / --max-batch-size	--max-num-seqs	개념 동일
배치 토큰 예산	--max-num-batched-tokens	--max-num-batched-tokens	키 이름 동일
최대 컨텍스트 길이	--max-model-len (버킷 상한)	--max-model-len	furiosa 는 버킷 범위 내
프리픽스 캐싱	--enable-prefix-caching	--enable-prefix-caching	둘 다 기본 ON
텐서 병렬 (TP)	빌드에 고정 (serve 에션 무시)	--tensor-parallel-size (기동 시)	★ 가장 큰 차이
메모리 점유율	(등가 없음) · --spare-blocks-ratio	--gpu-memory-utilization	NPU 엔 직접 등가 없음
NPU 전용	--devices npu:N · --npu-queue-limit · 버킷	—	하드웨어 종속

※ tp 는 furiosa 에션 빌드에 박혀 serve 플래그가 무시됨 (다음 장). pp·dp 는 serve 에서 변경 가능 .

furiosa serve = vLLM 플래그 미러링 + NPU 전용 추가

furiosa 는 vLLM 사용자가 그대로 옮겨올 수 있게 같은 플래그를 많이 채택했습니다.

양쪽 공통 (furiosa 가 vLLM 서 채택)

- | | |
|----------------------------|--------------------------------|
| ✓ --host / --port | ✓ --max-model-len |
| ✓ --max-num-batched-tokens | ✓ --enable-prefix-caching |
| ✓ --tensor-parallel-size * | ✓ --pipeline-parallel-size |
| ✓ --data-parallel-size | ✓ --reasoning-parser |
| ✓ --tool-call-parser | ✓ --structured-outputs-backend |
| ✓ --served-model-name | ✓ --api-key / --chat-template |
- * --tensor-parallel-size 는 plumbing 만 같고, prebuilt 아티팩트에선 무시됨

furiosa 전용 (vLLM 엔 없음)

- | | |
|--------------------------------------|----------------------|
| --devices npu:N | 어느 NPU 카드 / PE 에 올릴지 |
| --prefill-buckets / --decode-buckets | 정적 shape 버킷 지정 |
| --max-batch-size / --min-batch-size | NPU 배치 경계 |
| --npu-queue-limit | NPU 큐 깊이 |
| --spare-blocks-ratio | KV 예약 블록 비율 |
| --enable-jit-compilation | 기동 시 재컴파일 (opt-in) |
| --scheduler-kind / -loop-type | 네이티브 스케줄러 선택 |

실전에서 부딪히는 차이 5 가지

주로 furiosa 의 “정적 · 사전 컴파일” 성격에서 옵니다 .

1 tp 는 serve 에서 못 바꾼다

prebuilt 아티팩트는 tp 가 바이너리에 박혀 있어 --tensor-parallel-size 가 무시됨 (WARNING). 바꾸려면 그 값으로 build 재실행 . (vLLM 은 기동 플래그로 즉시 변경)

2 프롬프트 길이 = 버킷 상한

정적 shape 이라 최대 prefill 버킷을 넘는 프롬프트는 거부됨 . (vLLM 은 동적 shape 이라 --max-model-len 까지 자유)

3 gpu-memory-utilization 증가 없음

NPU 는 메모리 자동 프로파일링 노브가 없음 — --spare-blocks-ratio 가 가장 근접 . (vLLM 은 0.9 로 HBM 점유율 직접 조절)

4 첫 준비는 무겁고 , 기동은 가볍다

furiosa: build 한번 (수십 분) → serve 는 로드만 (빠른 · 결정적). vLLM: build 없음 → 매 기동마다 컴파일 / 캡처 비용 .

5 아티팩트 이동성

furiosa 아티팩트는 특정 모델 · tp · SDK 버전에 묶임 . vLLM 은 HF 체크포인트만 있으면 어느 GPU 에서나 동일 기동 .

정리 — 무엇을 언제 쓰나

furiosa-llm 이 맞는 경우

- Furiosa RNCD NPU 에 배포가 전제
- 모델 · 구성이 고정 — 한 번 빌드해 오래 서빙
- 빠르고 결정적인 기동, 사전 최적화가 중요
- AOT 로 굳힌 정적 파이프라인이 잘 맞는 워크로드

vLLM 이 맞는 경우

- GPU 환경 · 다양한 모델을 자주 바꿔 실험
- tp· 양자화 · shape 를 기동마다 유연하게
- 투기적 디코딩 · 전체 샘플링 기능이 필요
- 엔진 내부를 읽고 · 고치고 · 검증해야 함

한 줄 요약 furiosa-llm = 미리 컴파일해 열린 아티팩트 + 네이티브 런타임 (NPU) · vLLM = 기동마다 로드 · 컴파일하는 열린 Python 런타임 (GPU)